

Python Interview Coding Questions

Easy examples with simple explanations — beginner friendly

How to use these notes

First understand what the question is asking. Then understand the idea, type the code yourself, and finally try changing the input. For interviews, focus on explaining why each line is used.

1. Even or Odd

Explanation: Use % 2. If remainder is 0, the number is even.

Code:

```
num = 10
if num % 2 == 0:
    print("Even")
else:
    print("Odd")
```

2. Leap Year

Explanation: A simple beginner rule is: if the year is divisible by 4, it is a leap year. The complete calendar rule also handles centuries using 100 and 400.

Code:

```
year = 2024
if year % 4 == 0:
    print("Leap Year")
else:
    print("Not a Leap Year")
```

3. Reverse a String

Explanation: Python slicing[::-1] reads the string from the end to the beginning.

Code:

```
name = "Chanchal"
print(name[::-1])
```

4. Palindrome

Explanation: A palindrome reads the same forward and backward. Compare the original string with its reverse.

Code:

```
word = "madam"
if word == word[::-1]:
    print("Palindrome")
else:
    print("Not Palindrome")
```

5. Factorial

Explanation: Factorial multiplies all numbers from 1 to the given number. Example: $5! = 120$.

Code:

```
num = 5
fact = 1
```

```
for i in range(1, num + 1):
    fact = fact * i
print(fact)
```

6. Positive or Negative

Explanation: Check whether the number is greater than or equal to zero.

Code:

```
num = -5
if num >= 0:
    print("Positive")
else:
    print("Negative")
```

7. Largest of Three Numbers

Explanation: Compare the three values using if/elif/else.

Code:

```
a = 10
b = 30
c = 20

if a > b and a > c:
    print(a)
elif b > c:
    print(b)
else:
    print(c)
```

8. Swap Two Numbers

Explanation: Python allows direct swapping using tuple unpacking.

Code:

```
a = 10
b = 20
a, b = b, a
print(a)
print(b)
```

9. Sum of Digits

Explanation: Take the last digit using % 10 and remove it using // 10.

Code:

```
num = 1234
total = 0
while num > 0:
    digit = num % 10
    total = total + digit
    num = num // 10
print(total)
```

10. Reverse a Number

Explanation: Build the reverse by repeatedly taking the last digit.

Code:

```
num = 1234
reverse = 0
while num > 0:
    digit = num % 10
    reverse = reverse * 10 + digit
    num = num // 10
print(reverse)
```

11. Prime Number

Explanation: A prime number has exactly two factors: 1 and itself.

Code:

```
num = 7
if num > 1:
    for i in range(2, num):
        if num % i == 0:
            print("Not Prime")
            break
    else:
        print("Prime")
else:
    print("Not Prime")
```

12. Fibonacci Series

Explanation: Each new number is the sum of the previous two numbers.

Code:

```
a = 0
b = 1

for i in range(10):
    print(a)
    a, b = b, a + b
```

13. Armstrong Number

Explanation: For 153: $1^3 + 5^3 + 3^3 = 153$.

Code:

```
num = 153
total = 0
temp = num

while temp > 0:
    digit = temp % 10
    total = total + digit ** 3
    temp = temp // 10

if total == num:
    print("Armstrong")
else:
    print("Not Armstrong")
```

14. Count Vowels

Explanation: Check each character against a string containing vowels.

Code:

```
name = "chanchal"
count = 0

for i in name:
    if i in "aeiou":
        count = count + 1

print(count)
```

15. Count Words

Explanation: `split()` separates a sentence into words; `len()` counts them.

Code:

```
sentence = "Python is easy to learn"
words = sentence.split()
print(len(words))
```

16. Find Maximum in a List

Explanation: `max()` gives the largest value.

Code:

```
numbers = [10, 50, 20, 80, 30]
print(max(numbers))
```

17. Find Minimum in a List

Explanation: `min()` gives the smallest value.

Code:

```
numbers = [10, 50, 20, 80, 30]
print(min(numbers))
```

18. Sort a List

Explanation: `sort()` changes the original list into ascending order.

Code:

```
numbers = [5, 2, 8, 1]
numbers.sort()
print(numbers)
```

19. Remove Duplicates

Explanation: A set stores unique values. Convert it back to a list if needed.

Code:

```
numbers = [1, 2, 2, 3, 3, 4]
result = list(set(numbers))
print(result)
```

20. Find Common Elements

Explanation: Set intersection returns values present in both sets.

Code:

```
a = [1, 2, 3, 4]
b = [3, 4, 5, 6]
print(set(a) & set(b))
```

21. Find Duplicate Values

Explanation: `count()` can tell how many times a value occurs.

Code:

```
numbers = [1, 2, 3, 2, 4, 3]
duplicates = []

for i in numbers:
    if numbers.count(i) > 1 and i not in duplicates:
        duplicates.append(i)

print(duplicates)
```

22. Character Frequency

Explanation: A dictionary can store each character and its count.

Code:

```
name = "banana"
count = {}

for i in name:
    if i in count:
        count[i] += 1
    else:
        count[i] = 1

print(count)
```

23. Anagram

Explanation: Two strings are anagrams if they contain the same letters in a different order.

Code:

```
a = "listen"
b = "silent"

if sorted(a) == sorted(b):
    print("Anagram")
else:
    print("Not Anagram")
```

24. Missing Number

Explanation: For a small range, check which expected number is not in the list.

Code:

```
numbers = [1, 2, 3, 5]

for i in range(1, 6):
    if i not in numbers:
        print(i)
```

25. Second Largest

Explanation: A simple method is to sort the list and take the second-last item.

Code:

```
numbers = [10, 20, 50, 30]
numbers.sort()
print(numbers[-2])
```

26. Reverse a List

Explanation: Use slicing with [::-1].

Code:

```
numbers = [1, 2, 3, 4, 5]
print(numbers[::-1])
```

27. Merge Two Lists

Explanation: The + operator joins two lists.

Code:

```
a = [1, 2, 3]
b = [4, 5, 6]
print(a + b)
```

28. Even Numbers with List Comprehension

Explanation: List comprehension creates a new list in one line.

Code:

```
numbers = [1, 2, 3, 4, 5, 6]
result = [x for x in numbers if x % 2 == 0]
print(result)
```

29. Square Numbers

Explanation: Use ** 2 to calculate the square.

Code:

```
numbers = [1, 2, 3, 4]
result = [x ** 2 for x in numbers]
print(result)
```

30. Multiplication Table

Explanation: Use a loop from 1 to 10.

Code:

```
num = 5
for i in range(1, 11):
    print(num * i)
```

31. Star Pattern

Explanation: The string * operator repeats the star.

Code:

```
for i in range(1, 6):  
    print("'" * i)
```

32. Divisible by 3 and 5

Explanation: Use the logical operator and with two modulo checks.

Code:

```
for i in range(1, 101):  
    if i % 3 == 0 and i % 5 == 0:  
        print(i)
```

33. Remove Spaces

Explanation: `replace()` can replace spaces with an empty string.

Code:

```
name = "Hello Python"  
print(name.replace(" ", ""))
```

34. Convert String to List

Explanation: `list()` converts the string into individual characters.

Code:

```
name = "Python"  
print(list(name))
```

35. Convert List to String

Explanation: `join()` combines list items into one string.

Code:

```
words = ["Python", "is", "easy"]  
print(" ".join(words))
```

36. `map()`

Explanation: `map()` applies a function to every item. `lambda` makes a small function.

Code:

```
numbers = [1, 2, 3, 4]  
result = list(map(lambda x: x * 2, numbers))  
print(result)
```


37. filter()

Explanation: filter() keeps only items that satisfy a condition.

Code:

```
numbers = [1, 2, 3, 4, 5, 6]
result = list(filter(lambda x: x % 2 == 0, numbers))
print(result)
```

Important Questions to Practice Next

Basic: even/odd, leap year, factorial, prime, palindrome, Fibonacci, Armstrong, largest/smallest, swap numbers, sum/reverse digits.

Strings: reverse string, vowels, consonants, character frequency, duplicate characters, anagram, word count, remove spaces, first non-repeating character.

Lists: max/min, second largest, duplicates, remove duplicates, missing number, common elements, merge, sort, reverse, list comprehension.

Python concepts: functions, return, *args, **kwargs, lambda, map, filter, reduce, zip, enumerate, exception handling, iterators, generators, decorators, shallow/deep copy.

Interview Tip

Do not memorize the code line-by-line. Learn the pattern: condition → loop → update variable → print/return. Practice writing each program without looking at the answer.